Mini Project

Home Lab Monitoring

Ein Monitoring System, das aus verschiedenen Services besteht und alle relevanten Metrics im Homelab überwacht und Alerts verschickt.

Dienste im Monitoring System:

- Grafana
- Prometheus
- Loki
- Node Exporter (Windows & Linux)
- Promtail (veraltet) / Grafana Alloy

Sensoren:

- Temperatur
- Luftfeuchtigkeit

Output von Sensoren:

- LCD-Display
- Über Python Programm -> Dashboard (Grafana)

Sonstiges:

- Neues Github Repo
- Evtl. Homepage.dev verwenden für ein schönes Interface / Übersicht

• Light / Dark Mode anhand Lichtwerte

Chat GPT

Ja. Dann würde ich eure ursprüngliche Idee zu einem **vollwertigen „Smart Homelab Monitoring System“** ausbauen. Der Fokus bleibt klar auf dem Homelab – keine zusätzlichen Themenbereiche.

🏠 Smart Homelab Monitoring System

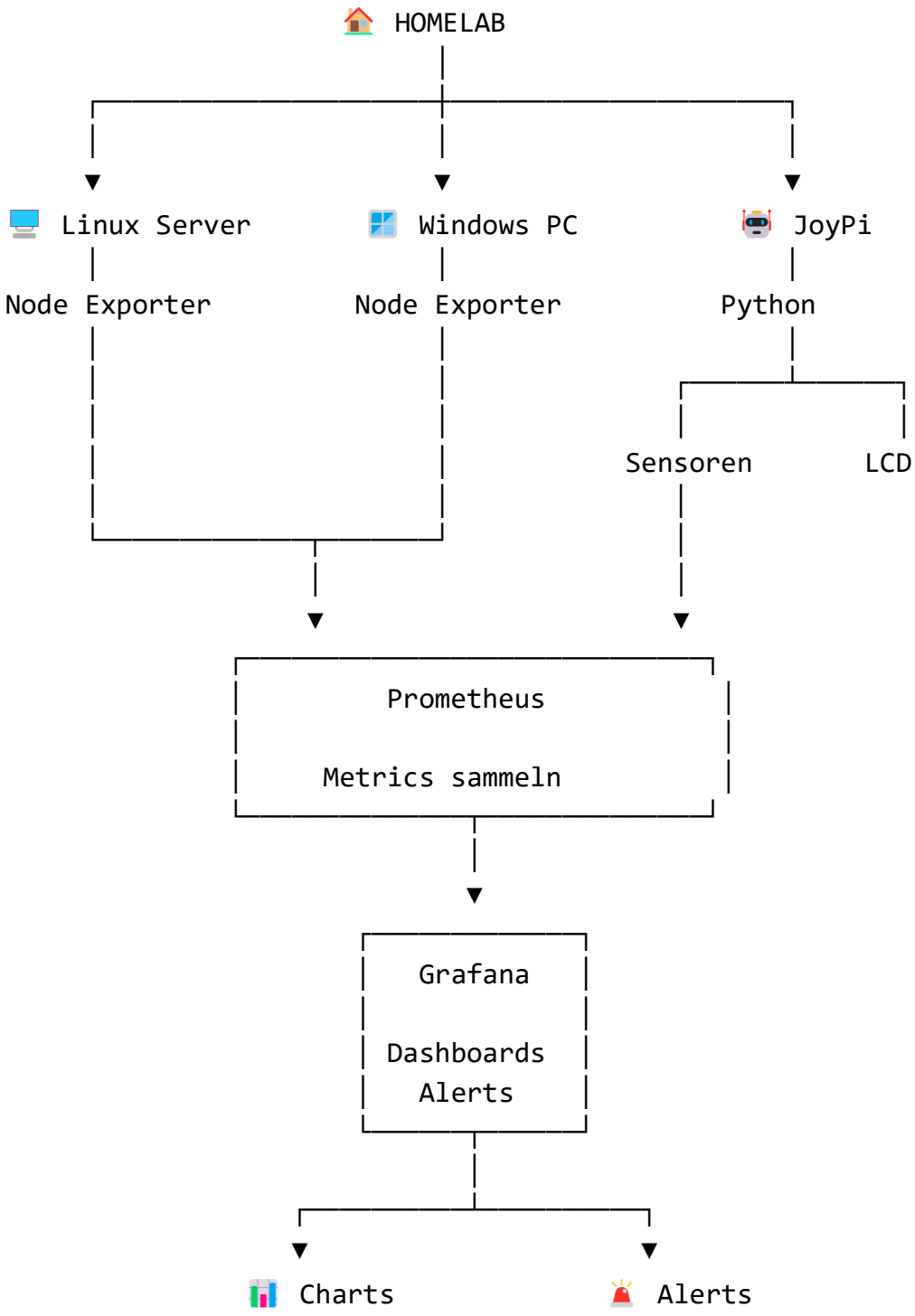
Grundidee

Wir entwickeln ein zentrales Monitoring-System, welches die **IT-Infrastruktur und die physische Umgebung eines Homelabs** überwacht.

Das System sammelt automatisch Metriken von Windows- und Linux-Systemen sowie Sensordaten des JoyPi. Die Daten werden zentral gespeichert, visualisiert und bei Problemen werden automatisch Alerts ausgelöst.

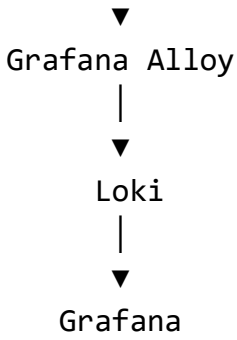
Damit habt ihr ein echtes **verteiltes Monitoring-System** und nicht nur ein einzelnes Python-Programm.

1. 🔬 Gesamtarchitektur



LOGGING

Linux / Windows
|



2. 🤖 JoyPi als Sensorstation

Der JoyPi wird eine eigene **Monitoring Node** im System.

Sensoren

Als Basis würde ich nehmen:

- 🌡️ **Temperatur**
- 💧 **Luftfeuchtigkeit**
- 💡 **Helligkeit**

Optional könnt ihr später erweitern:

- 📢 Geräuschpegel
- weitere Umweltsensoren

3. 💻 IT-Monitoring

Hier wird euer Projekt besonders interessant.

Mit **Node Exporter** überwacht ihr eure Rechner.

Linux / Windows

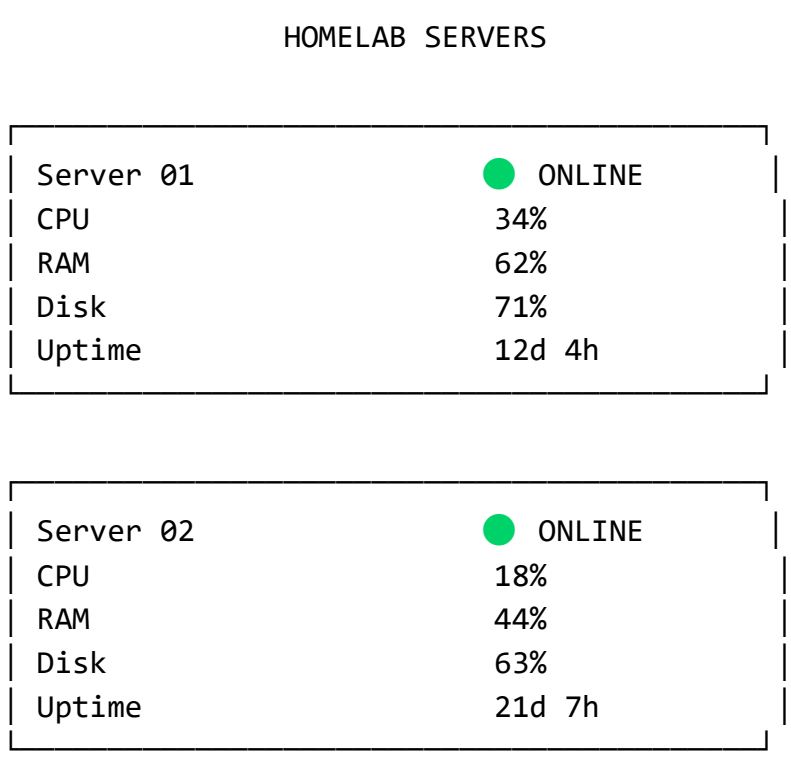
Beispielsweise:

- CPU-Auslastung

- RAM-Auslastung
- Festplattenbelegung
- Netzwerkverkehr
- System-Uptime
- Load
- verfügbare Ressourcen
- Systemstatus

Dadurch könnt ihr mehrere Geräte gleichzeitig überwachen.

Beispiel:



4. Prometheus

Prometheus ist eure zentrale Metrik-Sammlung.

Die verschiedenen Komponenten liefern Daten an Prometheus.

Zum Beispiel:

```
Linux Server
     |
```

```
      | CPU = 43%
      ▼
Node Exporter
      |
      ▼
Prometheus
```

Und gleichzeitig:

```
JoyPi
 |
 | temperature = 24.7
 ▼
Python Exporter
 |
 ▼
Prometheus
```

Prometheus hat dadurch einen zentralen Überblick über euer gesamtes Homelab.

5. 📊 Grafana

Grafana wird die zentrale Oberfläche.

Ich würde mindestens **drei Dashboards** erstellen.

Dashboard 1 – Homelab Overview

Die wichtigste Übersicht:

```
┌──────────────────────────────────────────────┐
│            🏠 HOMELAB OVERVIEW               │
├──────────┬──────────┬──────────┬─────────────┤
│ Servers  │ CPU      │ RAM      │ Storage     │
│  3 / 3   │ 42%      │ 61%      │ 72%         │
├──────────┴──────────┴──────────┴─────────────┤
│                                              │
│              CPU / RAM HISTORY               │
│                                              │
├──────────────┬───────────────────────────────┤
```

```
| Temperature      | Humidity                        |
| 24.3°C           | 48%                             |
|------------------|---------------------------------|
| Door             | Motion                          |
| 🔒 CLOSED        | No activity                     |
```

Dashboard 2 – Infrastructure

Detaillierte Informationen zu:

- CPU
- RAM
- Disk
- Netzwerk
- Uptime
- einzelne Server

Dashboard 3 – Environment

Für den JoyPi:

- Temperatur
- Luftfeuchtigkeit
- Helligkeit
- Tür
- Bewegung

6. 📜 Loki + Grafana Alloy

Hier könnt ihr zusätzlich **Logs überwachen**.

Anstatt nur zu wissen:

„Der Server hat gerade 90 % CPU.“

könnt ihr auch sehen:

„Was ist auf dem Server passiert?“

Dafür:

```
Windows / Linux
       │
       │ Logs
       ▼
Grafana Alloy
       │
       ▼
      Loki
       │
       ▼
    Grafana
```

In Grafana könnt ihr dann beispielsweise nach Fehlern suchen:

```
ERROR
WARNING
failed
connection
timeout
```

Damit habt ihr eine klare Trennung:

Prometheus → Metrics

Loki → Logs

Grafana → Visualisierung

7. 🚨 Alerting

Das würde ich zu einem zentralen Bestandteil eures Projekts machen.

Das System soll nicht nur Daten anzeigen, sondern **Probleme selbstständig erkennen**.

Beispiel: Temperatur

```
Temperature > 30°C
        ↓
```

```
    Prometheus
        ↓
    Alert Rule
        ↓
      Grafana
        ↓
   🚨 WARNING
```

Beispiel: Server offline

```
Node Exporter
      |
      X
nicht erreichbar
      |
      ▼
Prometheus
      |
      ▼
🚨 CRITICAL
Server01 is DOWN
```

Beispiel: Speicherplatz

```
Disk Usage > 90%
       ↓
   🚨 WARNING
       ↓
"Server01 storage almost full"
```

Beispiel: Tür

```
Door = OPEN
    +
ungewöhnliche Uhrzeit
    ↓
🚨 ALERT
"Homelab door opened"
```

8. 🖥️ LCD-Display

Das LCD des JoyPi kann als **lokale Statusanzeige** dienen.

Zum Beispiel:

```
┌────────────────────┐
│ HOMELAB STATUS     │
│                    │
│ Temp: 24.3 °C      │
│ Hum:  48 %         │
│ Status: OK         │
└────────────────────┘
```

Wenn ein Problem auftritt:

```
┌────────────────────┐
│ !! WARNING !!      │
│                    │
│ TEMP: 32.7 °C      │
│ TOO HIGH           │
└────────────────────┘
```

Damit habt ihr auch eine **physische Ausgabe** zusätzlich zum Grafana-Dashboard.

9. 🧠 Ein interessantes Extra: Homelab Health Score

Das würde ich als kreative Erweiterung einbauen.

Das System berechnet einen Gesamtstatus:

```
╔════════════════════════╗
║   HOMELAB HEALTH       ║
║                        ║
║        94 / 100        ║
║                        ║
║ 🟢 Servers      20/20  ║
```

```
╔═══════════════════════════╗
║ 🟢 Temperature   20/20    ║
║ 🟢 Humidity      20/20    ║
║ 🟡 Storage       17/20    ║
║ 🟢 Network       17/20    ║
╚═══════════════════════════╝
```

Wenn beispielsweise ein Server ausfällt:

```
HOMELAB HEALTH

       62 / 100

🔴 Servers       10/20
🟢 Temperature   20/20
🟢 Humidity      20/20
🟡 Storage       17/20
🟢 Network       15/20
```

Das gibt eurem Projekt einen eigenen Charakter und zeigt, dass ihr das System **selbstständig weiterentwickelt** habt.

10. 🔄 Automatisierte Reaktionen

Wenn ihr noch eine Stufe weitergehen wollt, kann das System nicht nur warnen, sondern reagieren.

Beispielsweise:

```
Temperature > 30°C
        ↓
      Alert
        ↓
   Python / JoyPi
        ↓
    Lüfter ON
        ↓
Temperature < 27°C
        ↓
    Lüfter OFF
```

Oder:

```
Disk > 90%
     ↓
Alert
     ↓
Grafana
     ↓
Notification
```

So habt ihr:

Monitoring → Detection → Alerting → Reaction

11. 🧪 Eure perfekte Live-Demo

Ich würde die Präsentation nicht einfach mit einem fertigen Dashboard starten.

Macht einen **Fehlerfall**.

Ausgangszustand

```
🟢 HOMELAB HEALTH: 100%

Temperature: 23°C
CPU: 35%
RAM: 52%
Disk: 68%
Servers: 3/3
```

Dann erzeugt ihr ein Problem.

Zum Beispiel:

CPU-Last künstlich erhöhen.

Grafana:

```
CPU
100% ──────────────────┐
                       │
```



Danach:

🚨 CPU usage above threshold

Dann könnt ihr einen zweiten Fehler erzeugen:

Temperatursensor erwärmen.

```
Temperature
24°C → 27°C → 30°C → 32°C
```

Das System:

🚨 `HIGH TEMPERATURE`

Und das LCD zeigt gleichzeitig:

```
!! WARNING !!
TEMP: 32.1°C
```

Das ist eine sehr starke Demonstration, weil ihr den kompletten Weg zeigen könnt:

Sensor → Python → Prometheus → Grafana → Alert → LCD

12. 🗀 GitHub-Repository

Für das neue Repository würde ich eine Struktur in diese Richtung wählen:

```
homelab-monitoring/
│
├── README.md
│
├── docs/
│   ├── architecture.md
│   ├── setup.md
│   └── screenshots/
│
├── joypi/
```

```
│   ├── sensors/
│   ├── lcd/
│   ├── exporter/
│   └── main.py
│
├── prometheus/
│   ├── prometheus.yml
│   └── rules/
│
├── grafana/
│   └── dashboards/
│
├── loki/
│   └── config/
│
├── alloy/
│   └── config/
│
├── docker-compose.yml
│
└── LICENSE
```

Das macht euer Repository auch für die Dokumentation und Präsentation übersichtlich.

🎯 Der endgültige Projektumfang

Ich würde eure Idee offiziell ungefähr so formulieren:

Smart Homelab Monitoring System

Entwicklung eines zentralen Monitoring-Systems zur Überwachung eines Homelabs. Das System erfasst sowohl technische Metriken von Windows- und Linux-Systemen als auch Umgebungsdaten über einen JoyPi. Prometheus dient zur Sammlung und Speicherung der Metriken, Grafana zur Visualisierung und zum Alerting und Loki zusammen mit Grafana Alloy zur zentralen Erfassung und Analyse von Logs. Die Sensordaten des JoyPi werden über ein Python-Programm in das Monitoring-System integriert und zusätzlich auf einem LCD dargestellt.

Kernkomponenten

🤖 Hardware

- JoyPi
- Temperatur
- Luftfeuchtigkeit
- Helligkeit
- Tür
- Bewegung
- LCD

🖥️ Monitoring

- Prometheus
- Grafana
- Node Exporter
- Loki
- Grafana Alloy

🐍 Integration

- Python
- Sensor-Auslesung
- Prometheus Metrics Endpoint

🚨 Funktionen

- Echtzeit-Monitoring
- historische Daten
- Dashboards
- Log-Analyse
- Alerting
- Health Score
- lokale LCD-Anzeige
- optional automatisierte Reaktionen

Damit habt ihr ein Projekt, das **klein anfangen kann, aber sehr viel Erweiterungspotenzial besitzt**. Und genau das ist für eure Bewertung interessant: Funktionalität, inhaltliche Tiefe, Kreativität und Produktkenntnis sind explizite Kriterien des Rasters.